

Singleton

1. Explicación del patrón

Singleton es un patrón de creación que garantiza que una clase tenga una única instancia y proporciona un punto de acceso global a ella.

La idea principal es controlar la creación de objetos de una clase para evitar que puedan existir varias instancias independientes.

En lugar de permitir que cualquier parte del programa cree objetos con `new`, la propia clase Singleton se encarga de crear y guardar su única instancia.

Después, cualquier cliente que necesite usar esa clase no crea un objeto nuevo, sino que solicita la instancia existente mediante un método común.

Por tanto, Singleton sirve cuando todos los clientes deben trabajar con el mismo objeto y compartir el mismo estado.

2. Cuándo se utiliza

Se utiliza el patrón Singleton cuando:

- Una clase debe tener una única instancia en todo el programa.
- Se necesita un único punto de acceso a esa instancia.
- Varios objetos o clases deben compartir la misma información.
- Se quiere controlar el acceso a un recurso común.
- Se quiere evitar que se creen varias copias de un objeto que representa algo único.
- El sistema necesita que todas las partes trabajen con el mismo estado.
- Se quiere centralizar la gestión de un recurso compartido.

Ejemplos típicos del material:

- Una única clase `Marcianos` que guarda los marcianos existentes en un juego.
- Un único objeto `Avion` que controla los asientos disponibles de un avión.

3. Cómo aplicarlo

Para aplicar Singleton se suelen definir estos elementos:

Clase Singleton

Es la clase que debe tener una única instancia.

Responsabilidades:

- Guardar una referencia estática a su propia instancia.
- Impedir que otras clases creen objetos directamente.
- Proporcionar un método de acceso global.

- Crear la instancia si todavía no existe.
- Devolver siempre la misma instancia cuando se solicita.

Instancia única

Es el único objeto real que existirá de la clase.

Suele guardarse en un atributo privado y estático.

Ejemplo conceptual:

```
private static Singleton instancia
```

Constructor privado

El constructor debe ser privado para evitar que otras clases puedan hacer:

```
new Singleton()
```

Así se obliga a que todos los clientes obtengan el objeto usando el método de acceso definido por la propia clase.

Método de acceso global

Es un método público y estático que devuelve la instancia única.

Ejemplos de nombres habituales:

- `getInstancia()`
- `getInstance()`
- `obtenerInstancia()`
- `operacionSingleton()`

Su funcionamiento suele ser:

1. Comprueba si la instancia ya existe.
2. Si no existe, la crea.
3. Devuelve la instancia guardada.
4. En llamadas posteriores, devuelve la misma instancia.

Cliente

Es cualquier clase que necesita usar el Singleton.

El cliente no crea el objeto directamente.

En vez de eso, pide la instancia mediante el método de acceso global.

Ejemplo conceptual:

```
Singleton s = Singleton.getInstancia()
```

4. Forma práctica de aplicarlo

Pasos:

1. Identificar una clase que deba existir una sola vez.
2. Crear dentro de esa clase un atributo privado y estático para guardar la instancia.
3. Hacer privado el constructor de la clase.
4. Crear un método público y estático que devuelva la instancia.
5. Dentro del método, comprobar si la instancia todavía no ha sido creada.
6. Si no existe, crearla usando el constructor privado.
7. Guardar la instancia creada en el atributo estático.
8. Devolver siempre esa misma instancia.
9. Sustituir las llamadas directas al constructor por llamadas al método de acceso.
10. Hacer que todos los clientes usen la misma instancia compartida.

5. Idea clave

El Singleton no permite crear varios objetos independientes de una misma clase.

La propia clase controla su única instancia y ofrece un punto de acceso común para todos los clientes.

Así, todos trabajan con el mismo objeto y con la misma información.

6. Ejemplos del material

Ejemplo de marcianos

En el material aparece un ejemplo de un juego donde la clase `Marcianos` se crea como Singleton.

El objetivo es que todas las clases del juego tengan la misma información sobre los marcianos existentes.

Situación:

- La clase `Marcianos` se instancia una sola vez.
- Cuando se instancia por primera vez, crea 10 marcianos.
- La clase `Jugador` debe matar la mayor cantidad de marcianos posible.
- La clase `Computador` crea marcianos cada vez que el jugador dispara.
- El ganador será quien más marcianos tenga.

Reparto indicado en el material:

- Grupo 1: clase `Marcianos`
- Grupo 2: clase `Jugador`
- Grupo 3: clase `Ordenador`

La clase `Marcianos` debe ser única porque representa el estado común de los marcianos en el juego.

Si hubiera varios objetos `Marcianos`, cada clase podría ver una cantidad distinta de marcianos, lo que rompería la lógica del juego.

Ejemplo de avión

En el material también aparece un ejemplo de control de asientos de un avión.

La clase principal es `Avion`, que debe instanciarse una sola vez.

Situación:

- Existe un único avión.
- Al crearse por primera vez, el avión tiene 180 asientos.
- La clase `Usuario` puede comprar billetes.
- La clase `Agencia` puede devolver o anular billetes.
- El programa principal debe tener un menú para elegir entre comprar o devolver billetes.

Clases del ejemplo:

- `Avion`
- `Usuario`
- `Agencia`

Operaciones del avión:

- `vendeBilletes(Billetes:int)`
- `devuelveBilletes(Billetes:int)`
- `cuantosAsientosQuedan():int`

La clase `Avion` debe ser Singleton porque todos los usuarios y agencias deben modificar el mismo número de asientos disponibles.

Si se crearan varios objetos `Avion`, cada uno tendría su propio contador de asientos y el sistema no reflejaría correctamente la disponibilidad real.

7. Ejemplo conceptual de uso

En vez de hacer esto:

```
Avion avion1 = new Avion()
```

```
Avion avion2 = new Avion()
```

se hace esto:

```
Avion avion = Avion.getInstance()
```

Después, tanto `Usuario` como `Agencia` usan esa misma instancia:

```
avion.vendeBilletes(2)
```

```
avion.devuelveBilletes(1)
```

Ambas operaciones afectan al mismo objeto `Avion`.

8. Ventajas prácticas

- Garantiza que solo exista una instancia de la clase.

- Proporciona un punto de acceso global.
- Permite compartir el mismo estado entre varias clases.
- Evita inconsistencias provocadas por tener varias copias de un mismo recurso.
- Centraliza el control de creación de la instancia.
- Puede inicializar el objeto solo cuando se necesita por primera vez.
- Es útil para recursos únicos o compartidos.

9. Cuidado

Singleton debe usarse con cuidado.

Aunque puede ser útil, también puede introducir problemas:

- Puede comportarse como una variable global.
- Puede aumentar el acoplamiento si muchas clases dependen directamente del Singleton.
- Puede ocultar un mal diseño si se usa para compartir datos sin control.
- Puede complicar las pruebas, porque el estado compartido puede mantenerse entre ejecuciones.
- En programas con varios hilos, hay que controlar que dos hilos no creen la instancia al mismo tiempo.
- Puede vulnerar el principio de responsabilidad única, porque la clase gestiona tanto su lógica normal como el control de su propia instancia.

No conviene usar Singleton simplemente para evitar pasar objetos como parámetros.

Debe reservarse para casos donde realmente tenga sentido que solo exista una instancia.

10. Diferencia práctica con otros patrones de creación

Factory Method, Abstract Factory y Prototype se centran en crear objetos de forma flexible.

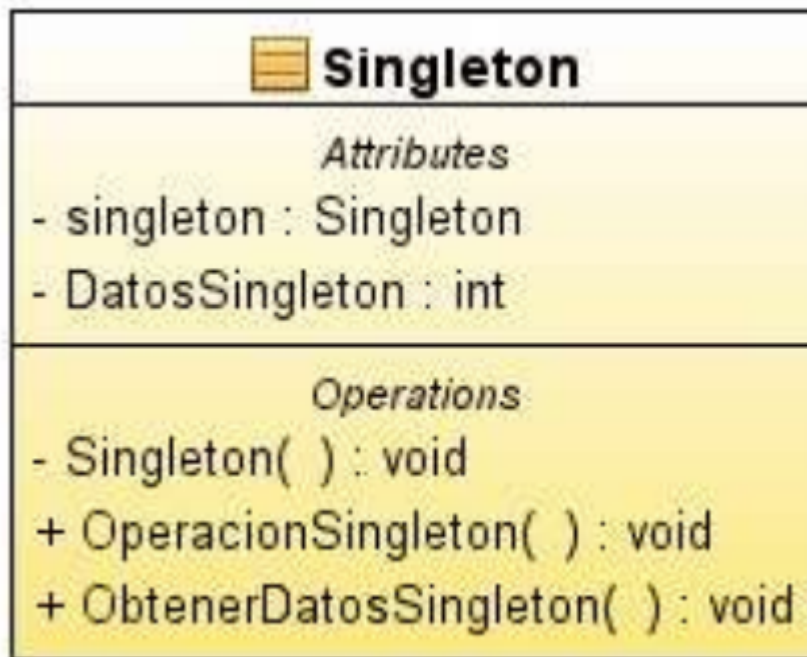
Singleton se centra en limitar la creación para que solo exista una instancia.

Por tanto:

- Factory Method decide qué producto concreto crear.
- Abstract Factory crea familias de productos relacionados.
- Prototype crea objetos clonando otros existentes.
- Singleton garantiza una única instancia y un acceso global a ella.

11. Plantilla

ries que se crea esta



12. Ejemplo

